CSC 402 · MOBILE APPLICATION PROGRAMMING · FIRST SEMESTER 2026–2027

Lab 4 — Your First Real Screen: Activities, Composables and State

Student Exercise File · Duration: 60 minutes · Week 4 · Paired tasks + individual assignment

Computer Science Department — College of Science and Humanities in Jubail

Section 1 Session Learning Objectives

By the end of this session you will be able to:

Obj. 1 Find the Activity in your project and point at the two lines in the manifest that make the app open when its icon is tapped.

Obj. 2 Look at a screen and **list its state** — the information that changes while the app runs.

Obj. 3 Write that state as a data class, and a **stateless** composable that takes it in and sends user actions out.

Obj. 4 Preview a screen in two different states without running the app.

Obj. 5 Wire a Route that owns the state, follow one tap all the way round, and make it fail on purpose.

Section 2 Before You Start

Software. Android Studio with the SDK installed, and one emulator or a physical device with USB debugging on.

Create the project. New Project → Empty Activity, then:

Field	Value
Name	Lab04Attendance
Package name	sa.edu.iau.cshj.lab04
Language	Kotlin
Build configuration language	Kotlin DSL
Minimum SDK	API 26 (Android 8.0)

Files you will work in: `AndroidManifest.xml`, `MainActivity.kt`, and one new file you create in Task 2.

Working arrangement. Tasks 1–4 in pairs at one machine; swap who types after each task. Section 4 is your own work and is graded individually.

Words you will need today

State — information that changes while the app runs and that the screen shows. **Stateless** — a composable that keeps none of its own. **Hoisting** — moving state up to whoever called the composable. **Route** — the composable that owns a screen's state.

All of these, and seven more, are on the glossary slide at the end of the lecture.

Section 3 Guided Tasks

Four tasks, in order. Each ends with a question — write your answer in the space given. Your instructor checks these during the session.

Task 1 — Find the way in — 10 minutes

Everything your app draws happens because Android opened an Activity. Find it, then prove what makes it the one that opens.

1.1 Open `app/manifests/AndroidManifest.xml`. Write the value of `android:name` on the `<activity>` element:

.MainActivity

1.2 Two lines inside `<intent-filter>` make this the Activity the launcher opens. Write both:

```
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
```

1.3 Open `MainActivity.kt`. Three things happen inside `onCreate`, in order. List them:

1. Calls super.onCreate().
2. Enables edge-to-edge display.
3. Sets the UI using setContent().

1.4 **Break it on purpose.** Comment out the `<category … LAUNCHER />` line, run the app, then look at the emulator's app drawer. What changed?

The app still installs, but its icon disappears from the app drawer because the LAUNCHER category was removed.

1.5 Put the line back and run again to confirm the icon returns. **Do not leave it commented out.**

Checkpoint 1

The app installed even with the launcher line removed. **Why?** Installing only needs a valid package. Being reachable from the home screen needs `MAIN` and `LAUNCHER` on one Activity.

Task 2 — Name the state, then write it down

10 minutes

This is the task the rest of the lab rests on. **Do not open Android Studio yet.** Read the screen description, then answer on paper.

The screen. An attendance card for one class session. It shows the session title, how many students have checked in so far, and one button. Tapping the button checks you in: the count goes up by one and the button becomes disabled.

2.1 List everything this screen **shows**:

Session title, checked-in count, and Check in button.

2.2 Of those, which ones **can change** while the app is running? Those are the state.

The checked-in count and whether the student has checked in (isCheckedIn), which controls if the button is enabled. The session title does not change.

2.3 What can the user **do** on this screen? Write each one as a name starting with `on`.

onCheckIn

2.4 Now open Android Studio. Create a package `ui` inside your package, then a file `AttendanceScreen.kt` inside it. Type in the class you just described:

```
data class AttendanceUiState(
   val sessionTitle: String = "",
   val checkedInCount: Int = 0,
   val isCheckedIn: Boolean = false
)
```

- Every field is a `val`, never a `var`. You will see why in Task 4.
- Every field has a default, so `AttendanceUiState()` on its own is a valid starting state. That is what makes the previews in Task 3 possible.

> **Checkpoint 2**
>
> You have just done the part of the job that has nothing to do with Kotlin: deciding what the screen's state **is**. You will do this for every screen you build for the rest of the course.

Task 3 — Write the screen, and see it without running it

15 minutes

Now the screen itself. It will hold **no state of its own** — everything it shows arrives as a parameter, and the one thing the user can do leaves as a lambda.

3.1 In the same file, type in the screen. **Two lines are missing** — the ones marked `TODO`.

```
@Composable
fun AttendanceScreen(
   uiState: AttendanceUiState,
   onCheckIn: () -> Unit,
   modifier: Modifier = Modifier
) {
   Column(
      modifier = modifier.fillMaxSize().padding(24.dp),
      verticalArrangement = Arrangement.spacedBy(16.dp)
   ) {
      Text(uiState.sessionTitle)

      // TODO 1: show the count, e.g.  Checked in: 12

      // TODO 2: a Button that reports the tap, and that switches off
      //         once this student has checked in
   }
}
```

- **TODO 1.** A `Text` that reads the count out of `uiState`. Kotlin puts a value inside a string with `${ }` — the pattern is in Appendix B.
- **TODO 2.** `Button(onClick = …, enabled = …)`. The click handler is the lambda the screen was **given**, not code of your own. The `enabled` value is read from `uiState`.
- Let Android Studio add the imports (Alt+Enter / Option+Enter). Use **material3**, not material.

3.2 At the bottom of the file, add a preview. Replace `Lab04AttendanceTheme` with your project's theme name if it differs.

```
@Preview(showBackground = true, name = "Not checked in yet")
@Composable
private fun AttendanceScreenPreview() {
   Lab04AttendanceTheme {
      AttendanceScreen(
         uiState = AttendanceUiState("CSC 402 - Week 4", 12, false),
         onCheckIn = {}
      )
   }
}
```

3.3 Open the **Split** or **Design** view and confirm it renders. You have not run the app.

3.4 Add a **second** preview named `"Already checked in"`. Only one argument changes.

3.5 Compare the two. What is different about the button, and which single field caused it?

The button is enabled in the first preview and disabled in the second because isCheckedIn changes from false to true.

Checkpoint 3

A screen that loaded its own data from a database could **not** be previewed like this — there is no database inside Android Studio. You can preview yours **because** of how you wrote the signature in step 3.1.

Task 4 — Make it work, then break it

15 minutes

Your screen draws but nothing happens when you tap. Nobody owns the count. In this task the **caller** takes ownership — state flows down to the screen, events flow up to the caller.

4.1 In the same file, add the Route. Type it exactly as shown:

```
@Composable
fun AttendanceRoute() {
   var uiState by remember {                        // Week 6 topic
      mutableStateOf(AttendanceUiState("CSC 402 - Week 4", 12))
   }

   AttendanceScreen(
      uiState = uiState,                            // state flows DOWN
      onCheckIn = {                                 // events flow UP
         uiState = uiState.copy(
            checkedInCount = uiState.checkedInCount + 1,
            isCheckedIn = true
         )
      }
   )
}
```

- `remember` and `mutableStateOf` are **Week 6** topics. Today just accept that they let the caller hold a value that Compose watches.
- You also need `getValue` and `setValue` imported, or `by` will not compile. The four lines are in Appendix B.

4.2 In `MainActivity.kt`, call your Route inside `setContent`:

```
setContent {
   Lab04AttendanceTheme {
      AttendanceRoute()
   }
}
```

4.3 Run the app. Tap **Check in** once. The count goes up by one and the button switches off.

4.4 Fill in the trace for the single tap you just made:

Step	What happens	Which function
1	The user taps the Button.	AttendanceScreen
2	The onCheckIn event is triggered.	AttendanceScreen
3	A new state is created using copy().	AttendanceRoute
4	The updated state is passed to the screen.	AttendanceRoute
5	The count increases and the button is disabled.	AttendanceScreen

4.5 **Break it on purpose.** Replace the body of the lambda with this, run, and tap:

```
onCheckIn = {
   uiState.checkedInCount + 1     // calculated, then thrown away
}
```

4.6 What do you see now, and which step of your trace failed?

The count stays at 12, and the button remains enabled because the state was not updated.
Step 3 failed because no new state was created and assigned.

4.7 Put the `copy` version back and confirm it works again.

Checkpoint 4 — show your instructor

Before you leave, show: (a) two working previews, (b) the app responding to one tap, (c) your completed trace, and (d) the frozen version from step 4.5.

Section 4 Individual Assignment — Semester App, Part 1

Read this first — the assignment runs all semester

From this lab onward, every lab ends with one individual task that adds to **the same app**. By the last lab you will have a small but complete Android application that is entirely your own work, built alongside — and separately from — your team's project.

The app is called the **Semester App**. It is graded in parts, one part per lab. Part 1 is due before the Week 5 lab.

Your Semester App: a Study Planner. A single-user app that keeps track of study tasks for your own courses. It stays small on purpose: a list of tasks, a way to add one, a way to mark one done, and a little polish.

Part 1 — the home screen, built the way this lab taught you.

P1.1 **On paper first.** Sketch your home screen. List what it shows, which of those things can change, and what the user can do. Keep this page — you will hand a photo of it in.

P1.2 Create a project named `StudyPlanner` (package `sa.edu.iau.cshj.studyplanner`, Kotlin, Compose, minimum SDK 26). **Keep this project all semester** — every later part builds on it.

P1.3 Write `PlannerUiState` as a data class holding, at minimum: a greeting or your name, the total number of study tasks, and how many are done. All `val`, all with defaults.

P1.4 Write a **stateless** `PlannerHomeScreen` taking `PlannerUiState`, at least one event lambda, and a `modifier` with a default, last. It shows the greeting, a progress line such as *3 of 7 done*, and one button.

P1.5 Write **two** `@Preview` functions showing two meaningfully different states — for example nothing done yet, and everything done.

P1.6 Write `PlannerHomeRoute` that owns the state and handles the button by producing a new state with `copy`. Call it from `setContent` in `MainActivity`.

P1.7 Check the manifest: one Activity, with `MAIN` and `LAUNCHER`, and a sensible `android:label`.

Rules.

- `PlannerHomeScreen` holds **no** state of its own. All state lives in `PlannerHomeRoute`.
- Exactly **one** Activity in the manifest.
- Material 3 only (`androidx.compose.material3`).
- Individual work. Discuss ideas freely; do not share code files.
- If you used an AI assistant, record it in your **AI Use & Disclosure Log** the same week, including what you changed afterwards. Using AI costs no marks; not disclosing it does.

Submit to Blackboard, *Semester App — Part 1*, before the Week 5 lab:

- A ZIP of the project, **or** a link to your GitHub repository.
- A photo of your paper sketch and state list from P1.1.
- Two screenshots of your previews, and one of the running app after the button has been tapped.

Marking — Part 1 (5 marks)

Criterion	Marks
The state list in P1.1 names what the screen shows and what can change	0.5
`PlannerUiState` is one immutable data class with defaults	1
`PlannerHomeScreen` is stateless: data in, event out as a lambda, modifier last with a default	1.5
Two working previews showing two different states	1
`PlannerHomeRoute` owns the state and produces a new state object on the event	0.5
Runs; one Activity correctly declared; app label set	0.5

Section 5 Submission and Assessment

During the session your instructor checks the four items in Checkpoint 4. These count as lab participation and are not separately marked.

Before the Week 5 lab submit Semester App Part 1 as described in Section 4.

Self-check before you submit. This is the same six-point list from the lecture, plus four lines specific to this week.

☐ **Stateless?** No `remember` and no `mutableStateOf` inside my screen composable.

☐ **Parameters?** Every value my screen shows arrived as a parameter — nothing is typed into the layout.

☐ **Events?** Every user action leaves as a lambda named `onSomething`.

☐ **Material 3?** All Compose imports say `androidx.compose.material3`.

- ☐ **Modifier?** `modifier: Modifier = Modifier` is the **last** parameter.
- ☐ **Preview?** Both previews render without running the app.
- ☐ My screen composable's name starts with a capital letter and reads like a thing, not an action.
- ☐ My Route produces a **new** state object with `copy` — it never edits the old one.
- ☐ The app has exactly one Activity, with `MAIN` and `LAUNCHER`.
- ☐ My AI Use & Disclosure Log is up to date for this week.

Section A Appendix A — The Semester App, lab by lab

Keep the same project all semester. Each part is small; together they make one complete application. Later parts assume the earlier ones work, so do not skip a part.

Part	Lab	What you add	Builds on
1	Lab 4	Home screen: state list, UI state class, stateless composable, two previews, a Route that owns the state	—
2	Lab 5	A second screen (task detail), Navigation Compose between the two, and one Intent to share a task	Part 1
3	Lab 6	Survive rotation with `rememberSaveable`; hoist the screen's state properly	Part 2
4	Lab 7	Show your tasks in a `LazyColumn` of small reusable item composables, in one Activity	Part 3
5	Lab 8	A `ViewModel` owns the state; add a loading state and one `LaunchedEffect`	Part 4
6	Lab 9	Remember the user's name and theme choice with Preferences DataStore	Part 5
7	Lab 10	Persist tasks in a Room database exposed as a `Flow`; add and delete for real	Part 6
8	Lab 11	Share your task list with other apps through a Content Provider	Part 7
9	Lab 12	Schedule a daily study reminder with WorkManager and post a notification	Part 8
10	Lab 13	Refactor the background work with Coroutines and Flow; add a Repository	Part 9
11	Lab 14	Attach a study location to a task and show it on a Compose map	Part 10
12	Lab 15	Unit-test the state holder, one Compose UI test, fix accessibility issues, prepare a release build	Part 11

> **Why one app across twelve parts**
>
> Because the second time you open a screen you wrote three weeks ago, you find out whether you designed it well. A stateless screen from Part 1 survives every part after it; one that owns its own state has to be rewritten.

Section B Appendix B — Quick reference

You want to...	Write

Model a screen's state	`data class XUiState(val a: String = "", val b: Int = 0)`
Declare a screen	`@Composable fun XScreen(uiState: XUiState, onY: () -> Unit, modifier: Modifier = Modifier)`
Show a value inside text	`Text("Total: ${uiState.someNumber}")`
Make a changed state	`uiState = uiState.copy(b = uiState.b + 1)`
Preview one state	`@Preview @Composable private fun XPreview() { MyTheme { XScreen(XUiState(…), onY = {}) } }`
Own state in the caller	`var uiState by remember { mutableStateOf(XUiState()) }`
Enter Compose from the Activity	`setContent { MyTheme { XRoute() } }`

The four imports Task 4 needs (paste these if `by` will not compile):

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
```

Sources. Google's official Android documentation: *Introduction to activities*; the `<activity>` manifest element reference; *Guide to app architecture*; *Architecting your Compose UI*; *State and Jetpack Compose* — all at developer.android.com. Essential reference: Forrester, Boudjnah, Dumbravan & Tigcal (2025), *How to Build Android Applications with Kotlin* (3rd ed.), Packt — Chapters 1–3.